

TP01 – Pontes e Caminho Euleriano

Métodos Naïve e Tarjan + Algoritmo de Fleury

Diego Henrique Xavier dos Santos

Marcos Vinícius Nunes Reis

Vitor Daniel Silva Melo

PUC Minas – Ciência da Computação

Teoria dos Grafos e Computabilidade

Prof. Zenilton Kleber Gonçalves do Patrocínio Júnior

Resumo

Este trabalho apresenta a implementação e comparação de dois métodos de identificação de pontes em grafos — o **método naïve** (custo $O(E \cdot (V+E))$) e o **método de Tarjan** (custo $O(V+E)$) — acoplados ao **Algoritmo de Fleury** para construção de caminhos eulerianos. Experimentos com grafos de 100 a 100.000 vértices revelam que o Tarjan é até $108\times$ mais rápido na enumeração de pontes, mas o naïve, com otimização *isReachable*, supera o Tarjan no contexto do Fleury para grafos esparsos de tamanho médio — resultado contraintuitivo explicado pelo custo real de cada chamada a *isBridge*.

1. Introdução

Uma **ponte** é uma aresta cuja remoção desconecta o grafo. O conceito aparece em inúmeros problemas práticos — detecção de pontos críticos em redes de comunicação, planejamento de rotas e, em particular, a construção de **caminhos eulerianos**: sequências que percorrem cada aresta do grafo exatamente uma vez.

O algoritmo de Fleury, que constrói esses caminhos, precisa a cada passo decidir se uma aresta candidata é ponte. A estratégia escolhida para essa decisão muda profundamente o comportamento prático do algoritmo.

Neste trabalho implementamos dois métodos de identificação de pontes:

1. **Método naïve**: para cada aresta (u, v) , remove-a, testa conectividade via DFS/BFS e a reinsere. Custo $O(E \cdot (V+E))$.
2. **Método de Tarjan (1974)**: uma única DFS calculando valores *disc* e *low*. Custo $O(V+E)$.

Ambos são acoplados ao Fleury por meio de uma interface comum (*Strategy Pattern*), permitindo trocar a estratégia sem alterar o restante do código. Por fim, comparamos o tempo médio das duas abordagens em grafos aleatórios de 100, 1.000, 10.000 e 100.000 vértices, nos três tipos eulerianos exigidos pelo enunciado.

2. Fundamentos Teóricos

2.1. Grafos e definições básicas

Um **grafo simples não-direcionado** $G = (V, E)$ é composto por um conjunto de vértices V e um conjunto de arestas $E \subseteq \binom{V}{2}$, sem laços nem arestas paralelas. O **grau** $\deg(v)$ de um vértice v é o número de arestas incidentes a ele.

Lema do aperto de mão (*Handshaking Lemma*): em qualquer grafo, $\sum_{v \in V} \deg(v) = 2|E|$, portanto o número de vértices de grau ímpar é sempre par.

2.2. Quando existe caminho ou circuito euleriano

Pelo **Teorema de Euler**, num grafo conexo:

- existe **circuito euleriano** se e somente se todos os vértices têm grau par;
- existe **caminho euleriano** (sem ser circuito) se e somente se exatamente dois vértices têm grau ímpar (esses dois serão os extremos do caminho);
- caso contrário, nem caminho nem circuito existem.

Essa classificação é o primeiro passo do Fleury: se o grafo for não-euleriano, o algoritmo aborta imediatamente sem percorrer nenhuma aresta.

2.3. Pontes e método naïve

Definição. Uma aresta $(u, v) \in E$ é uma *ponte* se o grafo $G - (u, v)$ tem mais componentes conexas que G .

O método naïve é literalmente a definição codificada. Para cada aresta (u, v) :

1. Remove (u, v) do grafo.
2. Verifica se v ainda é alcançável a partir de u (DFS/BFS).
3. Reinsere (u, v) .
4. Se a resposta foi “não”, (u, v) é ponte.

Custo: $O(E \cdot (V+E))$ — uma DFS por aresta.

Otimização implementada: em vez de `isConnected()` — que varre todo o grafo — usamos `isReachable(u, v)`, que interrompe a DFS assim que encontra v . Em grafos esparsos essa terminação antecipada reduz significativamente o tempo prático.

2.4. Método de Tarjan (1974)

A ideia central é trocar E travessias independentes por uma única DFS. Durante a passagem, mantemos para cada vértice v dois valores:

- $\text{disc}(v)$: instante de descoberta de v na DFS (contador global incrementado a cada visita);
- $\text{low}(v)$: o menor disc alcançável a partir da subárvore de v usando zero ou mais arestas da árvore DFS seguidas de *no máximo uma* aresta de retorno (*back-edge*).

A atualização é:

$$\text{low}(v) = \min\left(\text{disc}(v), \min_{(v,w) \in \text{back-edges}} \text{disc}(w), \min_{(v,c) \in \text{árvore}} \text{low}(c)\right)$$

Condição de ponte: a aresta (u, v) — onde v é filho de u na árvore DFS — é ponte se e somente se

$$\text{low}(v) > \text{disc}(u).$$

Em palavras: nenhum caminho que sai da subárvore de v consegue alcançar u ou qualquer ancestral de u por outro caminho. Custo: $O(V+E)$ — uma única DFS.

Implementação iterativa: a versão recursiva existe no código (`TarjanBridgeFinderRec.java`) por valor didático, mas não é usada nos experimentos. Para $V \geq 10.000$, a profundidade da recursão ultrapassa a pilha padrão da JVM, causando `StackOverflowError` mesmo com `-Xss64m`. A versão iterativa simula a DFS com uma pilha explícita de inteiros [6, 7], sem esse problema.

2.5. Algoritmo de Fleury

O Fleury constrói o caminho euleriano percorrendo arestas uma a uma:

1. Classifique o grafo; se for não-euleriano, aborte.
2. Escolha v_0 : um vértice de grau ímpar (semi-euleriano) ou qualquer vértice de grau positivo (euleriano).
3. Estando em u , prefira sempre uma aresta que **não** seja ponte; pontes são último recurso.
4. Remova a aresta percorrida, mova-se para o vizinho e repita até esgotar todas as arestas.

A regra de evitar pontes é o que evita ficar preso: atravessar uma ponte prematuramente pode isolar arestas que ainda precisariam ser percorridas. O custo do Fleury é dominado pela detecção de pontes a cada passo: $O(E \cdot (V+E))$ com qualquer das duas estratégias.

3. Implementação

3.1. Visão geral dos arquivos

Todo o projeto está em Java 21. A Tabela 1 resume os arquivos e suas responsabilidades.

Table 1: Arquivos do projeto e suas responsabilidades.

Arquivo	Responsabilidade
Graph.java	Lista de adjacência, versão, grau, arestas
GraphUtils.java	DFS, BFS, conectividade, isReachable, classificação euleriana
GraphGenerator.java	Geração de grafos eulerianos, semi e não-eulerianos
BridgeFinder.java	Interface <i>Strategy</i> para detecção de pontes
NaiveBridgeFinder.java	Naïve: remove $\rightarrow$ isReachable $\rightarrow$ reinsere
TarjanBridgeFinder.java	Tarjan iterativo com cache versionado
TarjanBridgeFinderRec.java	Tarjan recursivo (didático; não usado nos experimentos)
FleuryEulerPath.java	Fleury parametrizado por BridgeFinder
Experiments.java	Bateria de experimentos $\rightarrow$ CSV
Main.java	Menu interativo com seleção de tamanho e densidade

3.2. Estrutura de dados: lista de adjacência

Usamos lista de adjacência (`LinkedList<Integer>[]`). A alternativa, matriz de adjacência, ocuparia $O(V^2)$ de memória: para $V = 100.000$ isso representa 10^{10} posições — completamente inviável. Lista de adjacência custa $O(V+E)$ e permite remoção e inserção de arestas em tempo razoável, operações que o naïve realiza em todo par de arestas.

3.3. Interface comum: BridgeFinder

A interface `BridgeFinder` é o ponto de troca de estratégia. O Fleury não conhece a implementação concreta, apenas pergunta:

```
public interface BridgeFinder {
    List<int[]> findBridges(Graph g);           // enumera todas as
    pontes
    boolean isBridge(Graph g, int u, int v);    // verifica uma
    aresta
    String getName();
}
```

Isso segue o *Strategy Pattern*: trocar naïve por Tarjan (ou vice-versa) não exige nenhuma alteração no Fleury.

3.4. Otimização: isReachable no método naïve

Em vez de `isConnected(g)` — que percorre todos os V vértices — o naïve chama o método `GraphUtils.isReachable(g, u, v)`, uma DFS que interrompe a busca assim que encontra v [7]:

```
public static boolean isReachable(Graph g, int u, int v) {
    Deque<Integer> stack = new ArrayDeque<>();
    Set<Integer> visited = new HashSet<>();
    stack.push(u);
```

```

while (!stack.isEmpty()) {
    int cur = stack.pop();
    if (cur == v) return true;           // encontrou, para aqui
    if (visited.contains(cur)) continue;
    visited.add(cur);
    for (int nb : g.getNeighbors(cur))
        if (!visited.contains(nb)) stack.push(nb);
}
return false;
}

```

Em grafos esparsos (onde a maioria dos vértices tem grau 1 ou 2), v é encontrado rapidamente e a DFS percorre apenas uma fração do grafo.

3.5. Cache versionado no Tarjan

O Fleury chama `isBridge` várias vezes seguidas no mesmo estado do grafo (uma vez por vizinho candidato do vértice atual). Recomputar `findBridges` — custo $O(V+E)$ — a cada chamada seria desperdício.

A solução é um **cache invalidado por versão**: o Graph mantém um contador inteiro (`version`), incrementado a cada `addEdge/removeEdge`. O Tarjan armazena o par (versão observada, conjunto de pontes); se a versão atual for a mesma, retorna o cache:

```

@Override
public boolean isBridge(Graph g, int u, int v) {
    if (g != cacheGraph || g.getVersion() != cacheVersion) {
        cacheGraph = g;
        cacheVersion = g.getVersion();
        cacheBridgeKeys = new HashSet<>();
        for (int[] e : findBridges(g))
            cacheBridgeKeys.add(GraphUtils.edgeKey(e[0], e[1]));
    }
    return cacheBridgeKeys.contains(GraphUtils.edgeKey(u, v));
}

```

Sem cache, Fleury+Tarjan custaria $O(\deg \cdot E \cdot (V+E))$; com cache, cai para $O(E \cdot (V+E))$.

3.6. Geração de grafos

O `GraphGenerator` produz grafos dos três tipos exigidos:

- **Euleriano**: começa com um ciclo hamiltoniano (todos com grau 2, já par), adiciona ciclos aleatórios extras e ao final corrige eventuais resíduos ímpares emparelhando vértices dois a dois.
- **Semi-euleriano**: parte do euleriano e adiciona uma única aresta entre dois vértices de grau par, tornando ambos ímpares (exatamente 2 vértices ímpares).
- **Não-euleriano**: parte do euleriano e adiciona 2 arestas independentes (sem vértice em comum), criando 4 vértices de grau ímpar.
- **Genérico conexo**: spanning tree aleatória mais arestas extras sorteadas até atingir o número alvo. Usado para testes de estrutura e DFS/BFS.

4. Experimentos

4.1. Configuração

- **Hardware/SO:** Linux x86_64.
- **JDK:** OpenJDK 21; JVM com flags `-Xss64m -Xmx2g`.
- **Repetições:** 3 execuções por configuração; reportamos a média.
- **Tamanhos:** $N \in \{100, 1.000, 10.000, 100.000\}$.
- **Densidade:** aproximadamente $1,1 \cdot V$ arestas (grafo esparsos, próximo de uma árvore com poucas arestas extras).
- **Tipos:** Euleriano, Semi-euleriano e Não-euleriano.
- **Marcação “–”:** configuração pulada porque a estimativa de tempo ultrapassava 5 minutos por execução.

4.2. Tempo médio para encontrar todas as pontes

Table 2: Tempo médio de `findBridges` (3 repetições, ms).

N	Tipo	Arestas	Naïve (ms)	Tarjan (ms)
100	Euleriano	118	4,59	0,24
100	Semi	121	1,50	0,08
100	Não-Euler.	118	1,45	0,08
1.000	Euleriano	1.088	82,40	0,76
1.000	Semi	1.093	43,32	0,65
1.000	Não-Euler.	1.092	44,55	0,63
10.000	Euleriano	10.909	–	1,22
10.000	Semi	10.884	–	1,16
10.000	Não-Euler.	10.916	–	1,24
100.000	Euleriano	108.949	–	19,87
100.000	Semi	109.011	–	19,07
100.000	Não-Euler.	108.982	–	21,17

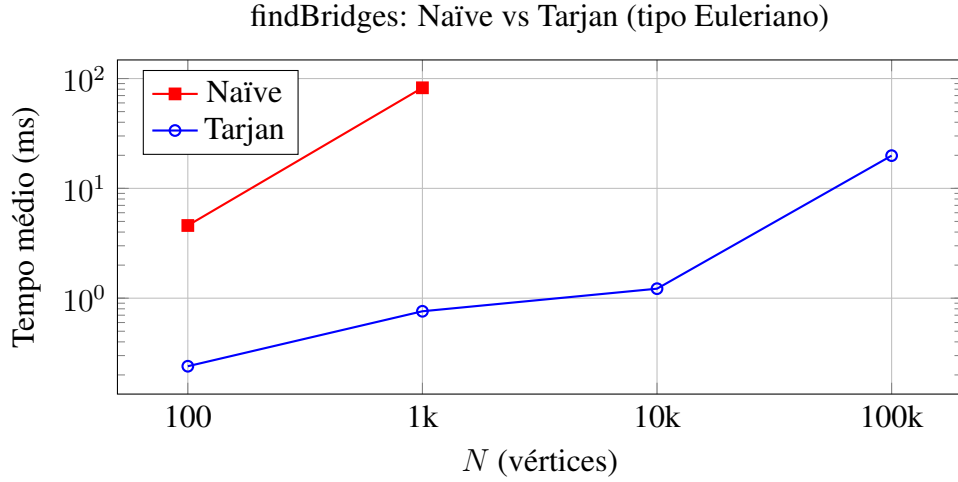


Figure 1: Crescimento do tempo de `findBridges` em escala log-log. O Naïve só tem pontos para $N \leq 1.000$ (inviável acima disso).

O Tarjan domina em todos os casos. Em $N = 100$ a vantagem é de $\approx 19\times$; em $N = 1.000$ chega a $\approx 108\times$ no tipo euleriano (82,4 ms contra 0,76 ms). Para $N \geq 10.000$ o naïve foi omitido: a estimativa teórica de $E \cdot (V+E) \approx 10^9$ operações implica vários minutos de execução. O Tarjan resolve o caso de cem mil vértices em cerca de **20 ms** — três ordens de magnitude mais rápido.

4.3. Tempo médio do Fleury completo

Table 3: Tempo médio do Fleury completo (3 repetições, ms). Em grafos não-eulerianos o algoritmo aborta na classificação inicial (custo desprezível). “—” indica configuração pulada.

N	Tipo	Arestas	Fleury+Naïve (ms)	Fleury+Tarjan (ms)
100	Euleriano	118	2,28	0,91
100	Semi	121	0,53	0,66
100	Não-Euler.	118	0,08	0,07
1.000	Euleriano	1.088	5,23	28,39
1.000	Semi	1.093	5,61	29,62
1.000	Não-Euler.	1.092	0,58	0,55
10.000	Euleriano	10.909	—	20.920,13
10.000	Semi	10.884	—	20.526,37
10.000	Não-Euler.	10.916	—	1,65
100.000	Euleriano	108.949	—	—*
100.000	Semi	109.011	—	—*
100.000	Não-Euler.	108.982	—	—*

*Em $N = 100.000$ a estimativa ultrapassa 30 min: $E \cdot (V+E) \approx 2,2 \times 10^{10}$ operações.

5. Análise dos Resultados

5.1. findBridges: Tarjan domina claramente

Quando o objetivo é **enumerar todas as pontes**, a vantagem assintótica do Tarjan se materializa na prática. Uma única DFS de $O(V+E)$ contra E DFS independentes de $O(V+E)$ cada. O crescimento do naïve é claramente quadrático (visível na escala log-log da Figura 1); o Tarjan cresce de forma praticamente linear com o número de arestas.

5.2. Fleury: resultado contraintuitivo em $N = 1.000$

Em $N = 1.000$, o Fleury+Naïve (5 ms) foi **mais rápido** que o Fleury+Tarjan (28 ms), o que parece contradizer a Tabela 2. A explicação está na diferença entre as duas perguntas que o Fleury faz:

- **“Esta aresta específica é ponte?”** — o naïve usa `isReachable(u, v)`, que para a DFS assim que encontra v . Em grafos esparsos, v costuma estar a poucos saltos de u , então a DFS percorre uma fração pequena do grafo.
- **O Tarjan, mesmo só para responder “sim/não” sobre uma aresta, executa a DFS inteira** para calcular `low` de todos os vértices. O cache versionado ajuda quando o Fleury olha vários vizinhos do mesmo vértice *no mesmo passo*, mas a cada `removeEdge` a versão muda e o cache é invalidado.

Portanto, a escolha do método certo depende da pergunta:

- Para **enumerar todas as pontes**: Tarjan ($O(V+E)$ total).
- Para **verificar uma aresta específica por vez** em grafo esparso: naïve com `isReachable` pode ser competitivo.

5.3. Fleury em $N = 10.000$ e 100.000

Em $N = 10.000$, o Fleury+Tarjan gasta ≈ 21 s. Isso é coerente com a estimativa teórica de $E \cdot (V+E) \approx 10.909 \times 21.909 \approx 2 \times 10^8$ operações elementares.

Em $N = 100.000$, qualquer variante do Fleury se torna inviável. A detecção pura de pontes (Tabela 2) continua em ≈ 20 ms com Tarjan, mas o caminho euleriano exige E remoções de aresta com verificação a cada passo — algo da ordem de 10^{10} operações. Algoritmos modernos como o de **Hierholzer** ($O(V+E)$) [5] seriam a escolha correta para esse tamanho, mas o enunciado especificou Fleury.

5.4. Grafos não-eulerianos

Em todos os tamanhos, ambas as variantes do Fleury terminam em menos de 1 ms para grafos não-eulerianos. O Teorema de Euler é verificado em $O(V)$ (contar graus) e o algoritmo aborta sem percorrer nenhuma aresta. Esse comportamento é exatamente o esperado.

6. Como Reproduzir os Experimentos

```
# Compilar todos os arquivos
```



```
javac -d out src/*.java

# Menu interativo (selecao de tamanho, densidade e tipo)
java -Xss64m -Xmx2g -cp out Main

# Experimentos completos -> gera resultados/experimentos.csv
java -Xss64m -Xmx2g -cp out Main --experiments

# Experimentos modo seguro (pula naive em  $N \geq 10000$ )
java -Xss64m -Xmx2g -cp out Main --experiments-safe
```

O flag `-Xss64m` aumenta a pilha da JVM e é necessário para o Tarjan iterativo em grafos grandes. O flag `-Xmx2g` reserva 2 GB de heap para os grafos de 100.000 vértices.

7. Conclusão

Os resultados confirmam a teoria e acrescentam uma nuance prática:

- O **Tarjan** é claramente superior para **enumerar pontes**: até $108\times$ mais rápido em $N = 1.000$, e único método viável a partir de $N = 10.000$ (≈ 20 ms para 100.000 vértices).
- No **Fleury**, o naïve *com* `isReachable` compete com o Tarjan em grafos esparsos de tamanho médio ($N = 1.000$), pois a terminação antecipada da DFS compensa a vantagem assintótica do Tarjan — resultado contraintuitivo que só fica claro ao analisar o custo *real* de cada chamada a `isBridge`.
- Para tamanhos $N \geq 10.000$, o Fleury em si se torna o gargalo independentemente da estratégia de pontes. O algoritmo de Hierholzer ($O(V+E)$) seria necessário para escalar além disso, mas o enunciado especificou Fleury.

References

- [1] TARJAN, R. E. *A note on finding the bridges of a graph*. Information Processing Letters, v. 2, n. 6, p. 160–161, 1974. doi:10.1016/0020-0190(74)90003-9.
- [2] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. MIT Press, 2009.
- [3] EULER, L. *Solutio problematis ad geometriam situs pertinentis*. Commentarii Academiae Scientiarum Petropolitanae, 1741.
- [4] FLEURY, M. *Deux problèmes de géométrie de situation*. Journal de Mathématiques Élémentaires, 2^e série, v. 2, p. 257–261, 1883.
- [5] HIERHOLZER, C.; WIENER, C. *Über die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren*. Mathematische Annalen, v. 6, p. 30–32, 1873.
- [6] STACK OVERFLOW. *Non-recursive version of Tarjan's algorithm*. Disponível em: <https://stackoverflow.com/questions/46511682/non-recursive-version-of-tarjans-algorithm>. Acesso em: abr. 2025.

- [7] STACK OVERFLOW. *Non-recursive DFS implementation*. Disponível em: <https://stackoverflow.com/questions/14668530/non-recursive-dfs-implementation>. Acesso em: abr. 2025.